



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

Aplicaciones para Comunicaciones en Red

TEMA:

**Práctica: Chat Multiusuario con Sockets No
Bloqueantes**

ALUMNO:

**Erreguin Franco Yair Alejandro
Boleta: 2022630121**

PROFESOR:

Moreno Cervantes Axel Ernesto

25 de junio de 2026

Índice

1. Introducción	2
1.1. Sockets No Bloqueantes y el Selector	2
1.2. Paradigma de Comunicación: Emulación de Multicast sobre TCP Unicast . .	2
1.3. Uso de Búferes (ByteBuffer de 2048 bytes)	3
1.4. Framing	3
1.4.1. Abstracción de una Sala	3
2. Desarrollo Técnico	4
2.1. Estructura del Servidor No Bloqueante	4
2.2. Estructura del Cliente No Bloqueante	9
3. Ejecución y Resultados	12
3.1. Monitoreo del Log de Estado del Servidor	12
3.2. Pruebas de Creación, Validación de Roles y Cierre	13
4. Conclusiones	15
5. Referencias	16

1. Introducción

El desarrollo de aplicaciones de chat multiusuario plantea el reto de gestionar múltiples conexiones simultáneas en el servidor de forma eficiente. En los esquemas tradicionales, cada cliente requiere un hilo dedicado en el servidor, lo que provoca un desperdicio masivo de recursos cuando los usuarios están inactivos.

El objetivo de esta práctica es diseñar e implementar un sistema de chat asíncrono utilizando la API de **Java NIO (New I/O)**. Esta arquitectura permite que un único hilo en el servidor gestione todas las conexiones, la creación y el borrado dinámico de salas de chat, así como el ruteo de mensajes utilizando sockets no bloqueantes y multiplexación de Entrada/Salida (E/S). A lo largo de este reporte, se detallarán los fundamentos de red que hacen posible esta solución, la estructura del protocolo diseñado por etiquetas y los resultados de las pruebas operativas.

1.1. Sockets No Bloqueantes y el Selector

En el modelo clásico de red, cuando un programa intenta leer un socket mediante `read()` y no hay datos disponibles, el hilo completo se congela (se bloquea) hasta que lleguen bytes desde la red.

Al configurar los canales en modo no bloqueante mediante `configureBlocking(false)`, se le indica directamente al sistema operativo que, si el búfer de recepción está vacío, no detenga el flujo del programa, sino que responda de inmediato con un valor de cero o un indicador de vacío. Para coordinar esto de manera inteligente, se implementa un **Selector**. El Selector actúa como un monitor centralizado que vigila todos los sockets abiertos. El hilo del servidor solo se despierta cuando el kernel del sistema operativo le notifica que algún cliente levantó la mano porque tiene datos listos para ser leídos o porque desea conectarse, optimizando al máximo el uso del procesador.

1.2. Paradigma de Comunicación: Emulación de Multicast sobre TCP Unicast

A nivel de infraestructura de red física, la multidifusión o *Multicast* real requiere del protocolo UDP y de routers configurados para duplicar los paquetes de forma nativa. Sin embargo, UDP no garantiza que los mensajes de texto o de control lleguen completos ni en orden, y las redes públicas de Internet suelen bloquear el tráfico multicast por seguridad.

Para solucionar esto, esta aplicación utiliza **TCP Unicast (Unidifusión)**. TCP garantiza que ningún mensaje se pierda y que todo llegue en el orden exacto en el que se envió. Como TCP es estrictamente una conexión punto a punto (1 a 1), el concepto de "salas de chat" se logra mediante una **Red Superpuesta en la Capa de Aplicación**. El cliente envía su mensaje únicamente al servidor (1 a 1). El servidor recibe el paquete, inspecciona sus diccionarios en memoria y se encarga de replicar el mensaje de forma secuencial, uno por uno, a los sockets de todos los usuarios inscritos en esa sala específica.

1.3. Uso de Búferes (ByteBuffer de 2048 bytes)

En Java NIO, los datos no se leen ni se escriben directamente en el canal, sino que pasan obligatoriamente por un contenedor intermedio llamado **ByteBuffer**. Para esta práctica, se definió un tamaño fijo de **2048 bytes (2 KB)** para el búfer de lectura.

Este tamaño se seleccionó estratégicamente en función de la **MTU (Unidad Máxima de Transmisión)** estándar de las redes de datos, la cual suele ser de 1500 bytes. Al asignar un búfer de 2 KB, garantizamos que cualquier mensaje de texto ordinario enviado en un paquete de red quepa holgadamente y pueda ser extraído por el servidor en una sola operación de lectura, evitando lecturas fragmentadas y saturación innecesaria en la memoria RAM.

1.4. Framing

Dado que TCP procesa la información como un flujo continuo de bytes sin separación intrínseca de mensajes, el programa implementa un protocolo personalizado basado en etiquetas para delimitar los paquetes y darles sentido en la capa de aplicación:

```
<tipo>msj<sala>s1<usuario>pepe<contenido>hola a todos
```

El servidor lee estos bloques de texto y utiliza un método de corte (**parseTag**) que localiza los caracteres < y > para extraer las variables limpias de forma determinista.

1.4.1. Abstracción de una Sala

A nivel físico de red, las salas no existen; son una ilusión creada en la memoria del servidor mediante tablas hash (**ConcurrentHashMap**). Una sala es simplemente una lista de referencias a sockets de clientes asociados a una palabra clave (el nombre de la sala). Cuando un usuario crea o se une a una sala, el servidor muta sus listas en memoria: remueve el socket de la sala anterior y lo anota en el conjunto de la nueva sala. Al hablar, el servidor solo busca esa lista en su agenda y le copia el mensaje a los sockets anotados.

2. Desarrollo Técnico

A continuación, se presentan los códigos fuente correspondientes al Servidor y al Cliente, estructurados bajo el enfoque asíncrono de Java NIO.

2.1. Estructura del Servidor No Bloqueante

El servidor gestiona el estado global (usuarios, salas y creadores) sobre un único hilo empleando colecciones concurrentes seguras y un registro dinámico en el selector de red.

```
1 import java.io.IOException;
2 import java.net.InetSocketAddress;
3 import java.nio.ByteBuffer;
4 import java.nio.channels.*;
5 import java.util.*;
6 import java.util.concurrent.ConcurrentHashMap;
7
8 public class NioChatServer {
9     private static final int PORT = 9015;
10    private final Selector selector;
11    private final ServerSocketChannel serverSocketChannel;
12
13    private final Map<SocketChannel, String> channelToUser = new
        ConcurrentHashMap<>();
14    private final Map<String, Set<SocketChannel>> rooms = new
        ConcurrentHashMap<>();
15    private final Map<String, String> roomToCreator = new
        ConcurrentHashMap<>();
16
17    public NioChatServer() throws IOException {
18        this.selector = Selector.open();
19        this.serverSocketChannel = ServerSocketChannel.open();
20        this.serverSocketChannel.bind(new InetSocketAddress(PORT));
21        this.serverSocketChannel.configureBlocking(false);
22        this.serverSocketChannel.register(selector, SelectionKey.OP_ACCEPT
        );
23
24        rooms.put("general", Collections.synchronizedSet(new HashSet<>()))
        ;
25        System.out.println("[ - ] Servidor TCP iniciado en el puerto " +
        PORT);
26        printServerStatus();
27    }
28
29    public void start() throws IOException {
30        while (true) {
31            if (selector.select() == 0) continue;
32
33            Iterator<SelectionKey> keys = selector.selectedKeys().iterator
            ();
34            while (keys.hasNext()) {
35                SelectionKey key = keys.next();
36                keys.remove();
```

```

37         if (!key.isValid()) continue;
38
39         if (key.isAcceptable()) {
40             handleAccept();
41         } else if (key.isReadable()) {
42             handleRead(key);
43         }
44     }
45 }
46
47 }
48
49 private void handleAccept() throws IOException {
50     SocketChannel clientChannel = serverSocketChannel.accept();
51     clientChannel.configureBlocking(false);
52     clientChannel.register(selector, SelectionKey.OP_READ);
53 }
54
55 private void handleRead(SelectionKey key) {
56     SocketChannel channel = (SocketChannel) key.channel();
57     ByteBuffer buffer = ByteBuffer.allocate(2048);
58     try {
59         int bytesRead = channel.read(buffer);
60         if (bytesRead == -1) {
61             disconnect(channel);
62             return;
63         }
64
65         String rawMessage = new String(buffer.array(), 0, bytesRead).
66             trim();
67         processProtocol(channel, rawMessage);
68     } catch (IOException e) {
69         disconnect(channel);
70     }
71 }
72
73 private void processProtocol(SocketChannel sender, String rawMessage)
74     throws IOException {
75     String tipo = parseTag(rawMessage, "tipo");
76     String usuario = parseTag(rawMessage, "usuario");
77     String salaName = parseTag(rawMessage, "sala");
78     String contenido = parseTag(rawMessage, "contenido");
79
80     switch (tipo) {
81         case "nuevo_usuario":
82             channelToUser.put(sender, usuario);
83             rooms.get("general").add(sender);
84             broadcastToRoom("general", "[SERVER] " + usuario + " entro
85                 al servidor.");
86             printServerStatus();
87             break;
88
89         case "crear_sala":

```

```

88         if (rooms.containsKey(salaName)) {
89             sendDirectMessage(sender, "[SERVER ERROR] La sala '" +
                salaName + "' ya existe.\n");
90         } else if (salaName.equalsIgnoreCase("general")) {
91             sendDirectMessage(sender, "[SERVER ERROR] Nombre
                reservado.\n");
92         } else {
93             rooms.put(salaName, Collections.synchronizedSet(new
                HashSet<>()));
94             roomToCreator.put(salaName, usuario);
95             sendDirectMessage(sender, "[SERVER] Sala '" + salaName
                + "' creada. Eres el admin.\n");
96             printServerStatus();
97         }
98         break;
99
100     case "borrar_sala":
101         if (!rooms.containsKey(salaName)) {
102             sendDirectMessage(sender, "[SERVER ERROR] La sala '" +
                salaName + "' no existe.\n");
103         } else if (salaName.equals("general")) {
104             sendDirectMessage(sender, "[SERVER ERROR] No puedes
                borrar la sala general.\n");
105         } else {
106             String creator = roomToCreator.get(salaName);
107             if (creator != null && creator.equals(usuario)) {
108                 Set<SocketChannel> members = rooms.get(salaName);
109                 broadcastToRoom(salaName, "[SERVER] Sala destruida
                    por su creador. Volviendo a general...");
110
111                 synchronized (members) {
112                     for (SocketChannel member : members) {
113                         rooms.get("general").add(member);
114                     }
115                 }
116                 rooms.remove(salaName);
117                 roomToCreator.remove(salaName);
118
119                 printServerStatus();
120                 broadcastToRoom("general", "[SERVER] La sala '" +
                    salaName + "' fue eliminada.");
121             } else {
122                 sendDirectMessage(sender, "[SERVER ERROR] Permiso
                    denegado. Solo el creador (" + creator + ")
                    puede borrarla.\n");
123             }
124         }
125         break;
126
127     case "ingresar":
128         if (!rooms.containsKey(salaName)) {
129             sendDirectMessage(sender, "[SERVER ERROR] La sala '" +
                salaName + "' no existe.\n");
130             return;

```

```

131         }
132         removeFromAllRooms(sender);
133         rooms.get(salaName).add(sender);
134
135         if (salaName.equals("general")) {
136             sendUserList(sender);
137         } else {
138             broadcastToRoom(salaName, "[SALA] " + usuario + " se
                unio a la sala.");
139         }
140         printServerStatus();
141         break;
142
143     case "msj":
144         if (rooms.containsKey(salaName) && rooms.get(salaName).
            contains(sender)) {
145             broadcastToRoom(salaName, "<" + usuario + "> " +
                contenido);
146         }
147         break;
148
149     case "salir":
150         if (rooms.containsKey(salaName)) {
151             rooms.get(salaName).remove(sender);
152             rooms.get("general").add(sender);
153             sendUserList(sender);
154             printServerStatus();
155         }
156         break;
157     }
158 }
159
160 private void printServerStatus() {
161     System.out.println("\n===== LOG DE SERVER
        =====");
162     System.out.println("[USUARIOS CONECTADOS (" + channelToUser.size()
        + ")]:" + channelToUser.values());
163     System.out.println("[SALAS ACTIVAS Y METADATOS:");
164     for (String room : rooms.keySet()) {
165         String owner = roomToCreator.getOrDefault(room, "Sistema");
166         int size = rooms.get(room).size();
167         System.out.println("    -> #" + room + " | Owner: " + owner + "
            | Miembros: " + size);
168     }
169     System.out.println("
        =====\n");
170 }
171
172 private void broadcastToRoom(String roomName, String message) throws
    IOException {
173     Set<SocketChannel> members = rooms.get(roomName);
174     if (members == null) return;
175     ByteBuffer buffer = ByteBuffer.wrap((message + "\n").getBytes());

```



```

176         synchronized (members) {
177             for (SocketChannel client : members) {
178                 if (client.isOpen()) {
179                     buffer.rewind();
180                     client.write(buffer);
181                 }
182             }
183         }
184     }
185
186     private void sendDirectMessage(SocketChannel client, String msg)
187         throws IOException {
188         if (client.isOpen()) {
189             client.write(ByteBuffer.wrap(msg.getBytes()));
190         }
191
192     private void sendUserList(SocketChannel receiver) throws IOException {
193         StringBuilder sb = new StringBuilder("[SALA GENERAL] Usuarios
194         globales:\n");
195         for (String user : channelToUser.values()) {
196             sb.append(" -> ").append(user).append("\n");
197         }
198         sendDirectMessage(receiver, sb.toString());
199     }
200
201     private void removeFromAllRooms(SocketChannel client) {
202         for (Set<SocketChannel> room : rooms.values()) {
203             room.remove(client);
204         }
205     }
206
207     private void disconnect(SocketChannel channel) {
208         try {
209             String user = channelToUser.remove(channel);
210             removeFromAllRooms(channel);
211             if (user != null) {
212                 printServerStatus();
213             }
214             channel.close();
215         } catch (IOException e) {
216             e.printStackTrace();
217         }
218     }
219
220     private String parseTag(String msg, String tag) {
221         String open = "<" + tag + ">";
222         int start = msg.indexOf(open);
223         if (start == -1) return "";
224         start += open.length();
225         int end = msg.indexOf("<", start);
226         if (end == -1) return msg.substring(start);
227         return msg.substring(start, end);

```

```

228
229     public static void main(String[] args) throws IOException {
230         new NioChatServer().start();
231     }
232 }
233 \

```

Listing 1: Código de NioChatServer.java

2.2. Estructura del Cliente No Bloqueante

El cliente emplea un hilo secundario síncrono para leer las entradas de teclado del usuario sin congelar la recepción asíncrona del búfer de red administrado por el selector principal. Asimismo, incorpora el comando explícito `/exit` para efectuar el cierre seguro de canales a nivel de sistema operativo.

```

1  import java.io.IOException;
2  import java.net.InetSocketAddress;
3  import java.nio.ByteBuffer;
4  import java.nio.channels.SelectionKey;
5  import java.nio.channels.Selector;
6  import java.nio.channels.SocketChannel;
7  import java.util.Iterator;
8  import java.util.Scanner;
9
10 public class NioChatClient {
11     private static final String HOST = "localhost";
12     private static final int PORT = 9015;
13     private SocketChannel socketChannel;
14     private Selector selector;
15     private String username;
16     private String currentRoom = "general";
17
18     public void start() throws IOException {
19         Scanner scanner = new Scanner(System.in);
20         System.out.print("Ingrese su nombre de usuario: ");
21         username = scanner.nextLine().trim();
22
23         selector = Selector.open();
24         socketChannel = SocketChannel.open(new InetSocketAddress(HOST,
25             PORT));
26         socketChannel.configureBlocking(false);
27         socketChannel.register(selector, SelectionKey.OP_READ);
28
29         sendPacket("nuevo_usuario", "", username, "");
30
31         new Thread(() -> {
32             System.out.println("\n--- INTERFAZ DISPONIBLE ---");
33             System.out.println(" /create <sala> -> Crea una sala");
34             System.out.println(" /delete <sala> -> Borra una sala (Solo si
35                 eres el due o)");
36             System.out.println(" /join <sala> -> Ingresa a una sala");

```

```

35     System.out.println(" /leave          -> Regresa a la sala
        general");
36     System.out.println(" /exit          -> Salir del programa");
37     System.out.println(" Escribe texto libre para chatear.\n");
38
39     while (true) {
40         String line = scanner.nextLine().trim();
41         if (line.isEmpty()) continue;
42
43         try {
44             if (line.startsWith("/create ")) {
45                 String targetRoom = line.substring(8).trim();
46                 sendPacket("crear_sala", targetRoom, username, "")
47                 ;
48             } else if (line.startsWith("/delete ")) {
49                 String targetRoom = line.substring(8).trim();
50                 sendPacket("borrar_sala", targetRoom, username, ""
51                 );
52             } else if (line.startsWith("/join ")) {
53                 String newRoom = line.substring(6).trim();
54                 currentRoom = newRoom;
55                 sendPacket("ingresar", currentRoom, username, "");
56             } else if (line.equals("/leave")) {
57                 sendPacket("salir", currentRoom, username, "");
58                 currentRoom = "general";
59             } else if (line.equals("/exit")) {
60                 System.out.println("[+] Cerrando aplicacion de
61                 forma segura...");
62                 socketChannel.close();
63                 System.exit(0);
64             } else {
65                 sendPacket("msj", currentRoom, username, line);
66             }
67         } catch (IOException e) {
68             System.out.println("[-] Error de conexion con el
69             servidor.");
70             break;
71         }
72     }
73     }).start();
74
75     while (true) {
76         if (selector.select() == 0) continue;
77         Iterator<SelectionKey> keys = selector.selectedKeys().iterator
78         ();
79         while (keys.hasNext()) {
80             SelectionKey key = keys.next();
81             keys.remove();
82
83             if (key.isReadable()) {
84                 handleRead();
85             }
86         }
87     }
88 }

```

```

83     }
84
85     private void handleRead() throws IOException {
86         ByteBuffer buffer = ByteBuffer.allocate(2048);
87         int bytesRead = socketChannel.read(buffer);
88         if (bytesRead == -1) {
89             System.out.println("[-] Desconectado del servidor.");
90             System.exit(0);
91         }
92         String msg = new String(buffer.array(), 0, bytesRead);
93         System.out.print(msg);
94     }
95
96     private void sendPacket(String tipo, String sala, String usuario,
97         String contenido) throws IOException {
98         StringBuilder packet = new StringBuilder();
99         packet.append("<tipo>").append(tipo);
100        if (!sala.isEmpty()) packet.append("<sala>").append(sala);
101        if (!usuario.isEmpty()) packet.append("<usuario>").append(usuario);
102        ;
103        if (!contenido.isEmpty()) packet.append("<contenido>").append(
104            contenido);
105
106        socketChannel.write(ByteBuffer.wrap(packet.toString().getBytes()))
107        ;
108    }
109
110    public static void main(String[] args) throws IOException {
111        new NioChatClient().start();
112    }
113 }

```

Listing 2: Código de NioChatClient.java

3. Ejecución y Resultados

3.1. Monitoreo del Log de Estado del Servidor

Al compilar y ejecutar los programas mediante los comandos `javac` y `java`, el servidor central imprime dinámicamente bloques estructurados de control. A continuación se muestra cómo se visualiza el log del servidor cuando se conectan de forma asíncrona los usuarios "Pepez "Maria", y Pepe decide crear de forma explícita la sala "s3":

```
===== LOG DE ESTADO (NIO) =====
[USUARIOS CONECTADOS (2)]: [Pepe, Maria]
[SALAS ACTIVAS Y METADATOS]:
  -> #general | Owner: Sistema | Miembros: 2
  -> #s1 | Owner: Pepe | Miembros: 0
=====
```



```
./init.sh
+ p5 git:(main) ls
+ compile.sh init.sh NioChatClient.class NioChatClient.java NioChatServer.class NioChats
+ p5 git:(main) ./compile.sh
+ p5 git:(main) ./init.sh
[-] Servidor iniciado en el puerto 9015

===== LOG DE ESTADO (NIO) =====
[USUARIOS CONECTADOS (0)]: []
[SALAS ACTIVAS Y METADATOS]:
  -> #general | Creador/Owner: Sistema | Miembros adentro: 0
=====

===== LOG DE ESTADO (NIO) =====
[USUARIOS CONECTADOS (1)]: [pepe]
[SALAS ACTIVAS Y METADATOS]:
  -> #general | Creador/Owner: Sistema | Miembros adentro: 1
=====

===== LOG DE ESTADO (NIO) =====
[USUARIOS CONECTADOS (2)]: [maria, pepe]
[SALAS ACTIVAS Y METADATOS]:
  -> #general | Creador/Owner: Sistema | Miembros adentro: 2
=====

===== LOG DE ESTADO (NIO) =====
[USUARIOS CONECTADOS (2)]: [maria, pepe]
[SALAS ACTIVAS Y METADATOS]:
  -> #general | Creador/Owner: Sistema | Miembros adentro: 2
  -> #s1 | Creador/Owner: pepe | Miembros adentro: 0
=====

java NioChatClient
+ ~ cd Desktop/redes/p5
+ p5 git:(main) java NioChatClient
Ingrese su nombre de usuario: pepe

--- INTERFAZ DISPONIBLE ---
/create <sala> -> Crea una sala (Seras el dueño)
/delete <sala> -> Borra una sala (Solo si eres dueño)
/exit -> Cerrar sesion.
/join <sala> -> Ingresa a una sala
/leave -> Regresa a la sala general
Escribe texto libre para mandar mensajes a tu sala actual.

[SERVER] pepe entro al servidor.
[SERVER] maria entro al servidor.
/create s1
[SERVER] Sala 's1' creada. Eres el administrador.

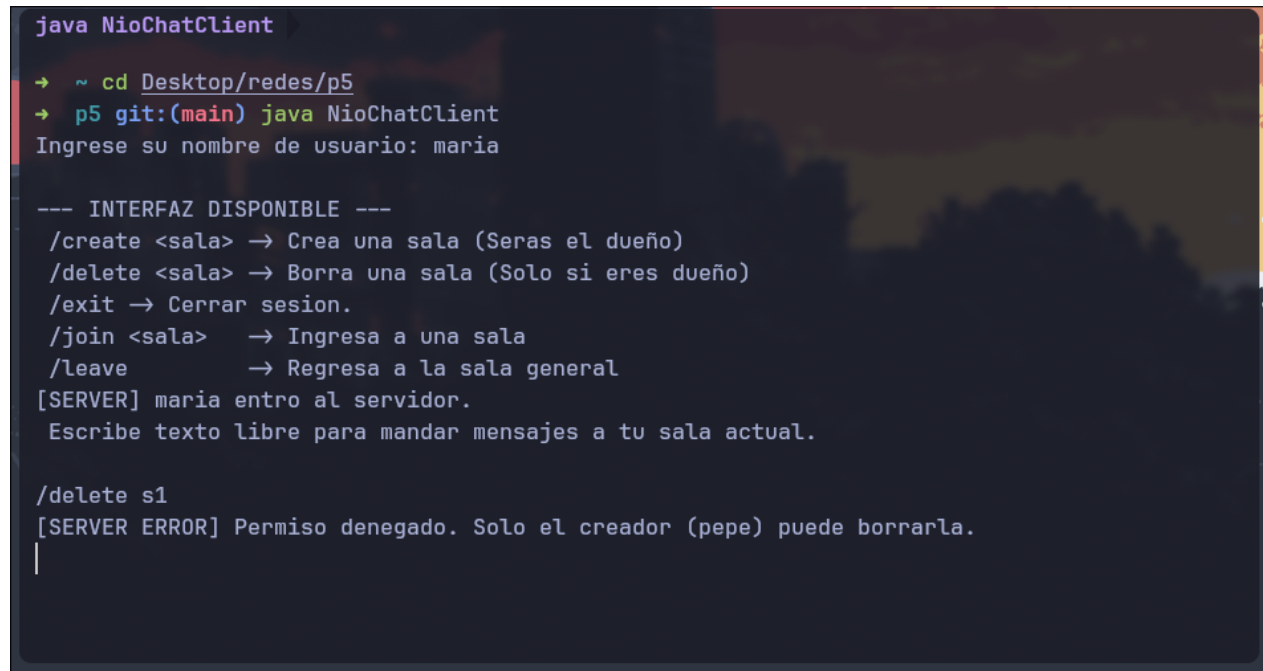
java NioChatClient
+ ~ cd Desktop/redes/p5
+ p5 git:(main) java NioChatClient
Ingrese su nombre de usuario: maria

--- INTERFAZ DISPONIBLE ---
/create <sala> -> Crea una sala (Seras el dueño)
/delete <sala> -> Borra una sala (Solo si eres dueño)
/exit -> Cerrar sesion.
/join <sala> -> Ingresa a una sala
/leave -> Regresa a la sala general
[SERVER] maria entro al servidor.
Escribe texto libre para mandar mensajes a tu sala actual.
```

Figura 1: Log activo en la consola del servidor mostrando el ruteo de usuarios y la persistencia de metadatos de las salas.

3.2. Pruebas de Creación, Validación de Roles y Cierre

Durante el ciclo operativo de las salas lógicas, si la usuaria **Maria** intenta enviar el comando de destrucción `/delete s3`, el servidor detecta que el canal emisor no coincide con el dueño registrado (**Pepe**), rechazando la solicitud de inmediato sin alterar la topología de red.



```
java NioChatClient
→ ~ cd Desktop/redes/p5
→ p5 git:(main) java NioChatClient
Ingrese su nombre de usuario: maria

--- INTERFAZ DISPONIBLE ---
/create <sala> → Crea una sala (Seras el dueño)
/delete <sala> → Borra una sala (Solo si eres dueño)
/exit → Cerrar sesion.
/join <sala> → Ingresa a una sala
/leave → Regresa a la sala general
[SERVER] maria entro al servidor.
Escribe texto libre para mandar mensajes a tu sala actual.

/delete s1
[SERVER ERROR] Permiso denegado. Solo el creador (pepe) puede borrarla.
|
```

Figura 2: Consola del cliente reflejando el rechazo de permisos cuando un usuario no administrador intenta borrar una sala activa.

Cuando el creador legítimo ejecuta el comando de borrado, los miembros reciben una notificación de desalojo en tiempo real y son redirigidos de regreso al canal de la sala **general**, la cual actualiza y lista de forma automática los usuarios globales conectados. Finalmente, al escribir el comando `/exit`, el cliente rompe la sesión de forma segura y el servidor libera el descriptor del socket de inmediato.

```

java NioChatClient

--- INTERFAZ DISPONIBLE ---
/create <sala> → Crea una sala (Seras el dueño)
/delete <sala> → Borra una sala (Solo si eres dueño)
/exit → Cerrar sesion.
/join <sala> → Ingresa a una sala
/leave → Regresa a la sala general
Escribe texto libre para mandar mensajes a tu sala actual.

[SERVER] pepe entro al servidor.
[SERVER] maria entro al servidor.
/create s1
[SERVER] Sala 's1' creada. Eres el administrador.
/join s1
[SALA] pepe se unio a la sala.
/delete s1
[SERVER] Esta sala ha sido destruida por su creador (pepe). Volviendo a general...
[SERVER] La sala 's1' fue eliminada de la topologia global.
|

java NioChatClient

--- INTERFAZ DISPONIBLE ---
/create <sala> → Crea una sala (Seras el dueño)
/delete <sala> → Borra una sala (Solo si eres dueño)
/exit → Cerrar sesion.
/join <sala> → Ingresa a una sala
/leave → Regresa a la sala general
[SERVER] maria entro al servidor.
Escribe texto libre para mandar mensajes a tu sala actual.

/delete s1
[SERVER ERROR] Permiso denegado. Solo el creador (pepe) puede borrarla.
/join s1
[SALA] maria se unio a la sala.
[SALA] pepe se unio a la sala.
[SERVER] Esta sala ha sido destruida por su creador (pepe). Volviendo a general...
[SERVER] La sala 's1' fue eliminada de la topologia global.
|

```

Figura 3: Visualización del canal de datos enviando mensajes de chat planos replicados dinámicamente por el servidor.

4. Conclusiones

El desarrollo de esta práctica me permitió comprender de manera directa los beneficios de la programación de red asíncrona y los sistemas basados en eventos. La migración de un modelo bloqueante tradicional a una arquitectura no bloqueante en Java NIO demuestra cómo optimizar radicalmente el uso de recursos del sistema operativo. Al emplear un único hilo central coordinado por un Selector, el servidor es capaz de delegar las esperas de red al kernel mediante mecanismos de bajo nivel como `epoll`, eliminando por completo la necesidad de mantener miles de hilos en espera pasiva.

Asimismo, la implementación me ayudó a aclarar la diferencia entre la multidifusión física(UDP) y lógica(Secuencial con TCP). Quedó demostrado que, para asegurar la consistencia de comandos de control cruciales (como la creación y el borrado seguro de salas bajo reglas de propiedad), el protocolo TCP ofrece el respaldo necesario frente a la inestabilidad de UDP. El diseño nos fuerza a entender que conceptos cotidianos de software como “una sala de chat.” o “un mensaje privado” no son más que abstracciones e ilusiones lógicas controladas eficientemente mediante tablas hash en la memoria volátil del servidor central.

5. Referencias

Referencias

- [1] Oracle, “Java standard channels and Selectors (Java SE docs),” 2025. [En línea]. Disponible: <https://docs.oracle.com/en/java/javase/>. [Accedido: 24-jun-2026].
- [2] Michael Kerrisk, “epoll(7) - Linux manual page,” Linux Man Pages, 2024. [En línea]. Disponible: <https://man7.org/linux/man-pages/>. [Accedido: 24-jun-2026].
- [3] Mozilla Developer Network, “Understanding Application Layer Routing over TCP/IP Streams.” [En línea]. Disponible: <https://developer.mozilla.org/>. [Accedido: 24-jun-2026].